4. La pile d'activités contient les éléments suivants : A, B, C (C au sommet). C fait un *setResult()*. Comment devient la pile ?

- ○ A. *A, B*
- ○ B. *A, B, B*
- ○ C. *A, B, C, C*
- ○ D. *A, B, C, B*

5. Dans quelle méthode du cycle de vie d'une activité, doit-on arrêter les traitements et les services exécutés par l'application ?

- ○ A. *onDestroy()*
- ○ B. *onResume()*
- ○ C. *onPause()*
- ○ D. *onStop()*

6. Dans le cas de rotation de l'écran, quelle méthode du cycle de vie est appelée sur l'activité en cours ?

- ○ A. *onCreate()*
- ○ B. *onRestart()*
- ○ C. *onPause()*
- ○ D. *onResume()*

7. Un objet *Bundle* ne peut sauvegarder que l'état des objets graphiques :

- ○ A. *Visibles sur l'écran*
- ○ B. *Ayant un ID*
- ○ C. *Créés dynamiquement à l'exécution*
- ○ D. *Présents dans le layout XML*

8. Quel paramètre d'intent permet de préciser le type MIME d'une ressource ?

- ○ A. *Category*
- ○ B. *Action*
- ○ C. *Data*
- ○ D. *Extras*

9. Que se passe-t-il si une activité cible n'est pas déclarée dans *AndroidManifest.xml* ?

- ○ A. *L'activité est lancée normalement*
- ○ B. *Android la déclare automatiquement*
- ○ C. *L'application plante (exception)*
- ○ D. *L'intent associé devient implicite*

10. Qu'est-ce qui caractérise principalement un intent explicite ?

- ○ A. *Actions, Catégories et Données*
- ○ B. *Contexte, Composant cible*
- ○ C. *Bundle, Extras*
- ○ D. *Schéma, Information, Requête, Fragment*

555F

AVIS IMPORTANT AUX ETUDIANTS

1. Chacune des feuilles de votre copie doit comporter une étiquette code à barres placée à l'endroit indiqué «coller ici votre code à barres».
2. Une copie d'examen comporte une seule «feuille principale» et des «feuilles suites». Sur chacune de vos feuilles, le code à barres est obligatoire.
3. Cette feuille d'examen est strictement personnelle. Elle ne doit comporter aucun signe distinctif. Elle doit être écrite en noir et/ou bleu.
4. Le non respect de l'une de ces recommandations peut faire attribuer la note ZERO à l'épreuve.

NOTE

Coller ici votre code à barre

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20

00	25	50	75

Institut Supérieur d'Informatique et de Mathématiques de Monastir

Année Universitaire 2025-2026

EXAMEN – Session de Rattrapage

Classe	Matière	Enseignant
L3 - INFO	Développement Mobile	Dr. Naoufel KHAYATI

Date	Mercredi 28 Janvier 2026	**Nombre de Pages**	7
Durée	1H30 (10h30 – 12h00)	**Documents**	Non Autorisés
Barème	Ex1 : 6,5 pts ; Ex2 : 2,5 pts ; Ex3 : 11 pts	**Durée estimée / exercice**	Ex1 : 30 min ; Ex2 : 10 min ; Ex3 : 50 min

NB : Dans cet examen, le langage à utiliser est Kotlin (Tout code écrit en Java sera considéré comme FAUX).

Exercice 1 (*6,5 pts*)

1. Soit une activité nommée ***LoginActivity*. (0,5)**
   - Comment s'appelle le fichier de **layout** associé à cette activité ?
   - Comment s'appelle le fichier de **binding** associé à cette activité ?

2. Lister quatre méthodes du cycle de vie d'une activité **(1)**

(a) ……………………… (b) ………………………

(c) ……………………… (d) ………………………

3. La résolution d'intents se base sur un test de conformité d'un intent avec un intent-filter. **(1)**
   - Dans quel fichier sont définis ces filtres d'intents ?
   - Ce test de conformité se base sur trois critères. Lesquels ?

(a) ……………………… (b) ……………………… (c) ………………………

4. Donner les lignes de code à écrire dans le fichier **build.gradle** afin d'activer le **ViewBinding** dans une application Android. **(1)**

5. Donner les instructions nécessaires afin de passer un appel direct au numéro « **123** » **via un intent implicite. (1,5)**

6. Donner les instructions nécessaires afin de lancer l'application de messagerie pour envoyer un SMS ayant le message « **Bonne Chance** » au numéro « **321** » **via un intent implicite. (1,5)**

Exercice 2 (*2,5 pts*)

1. Quelle activité est placée au sommet de la pile d'activités Android ?

○ A. *La première activité lancée*
○ B. *L'activité en arrière-plan*
○ C. *L'activité actuellement visible sur l'écran*
○ D. *L'activité racine*

2. Que se passe-t-il lorsqu'on appuie sur le bouton « Back » ?

○ A. *L'application entière se ferme*
○ B. *L'activité courante est détruite et retirée définitivement de la pile*
○ C. *L'activité courante est passée au fond de la pile*
○ D. *L'activité passe en pause uniquement*

3. La pile d'activités contient les éléments suivants : A, B, C (C au sommet). C lance l'activité B avec *startActivity()*. Comment devient la pile ?

○ A. *A, B*
○ B. *A, B, B*
○ C. *B*
○ D. *A, B, C, B*

SUITE

Coller ici votre
code à barre

Exercice 2 (*11 pts*)

NB : Pour cet exercice, le ViewBinding est déjà activé (L'objet associé est nommé « binding »).

Soit une application Android qui démarre sur une activité d'authentification (**LoginActivity**) demandant un nom d'utilisateur et un mot de passe pour pouvoir passer à l'activité d'accueil dite « **HomeActivity** ».
LoginActivity contient deux objets EditText et un bouton d'IDs respectifs ***edUsername***, ***edPw*** et ***btnLogin***.
Afin de valider les données de connexion, le login et le mot de passe doivent être identiques. Ainsi, deux cas se posent :

- Si les données saisies sont valides, l'utilisateur passe à « **HomeActivity** ». Dans ce cas, ***l'activité d'authentification doit transmettre le nom d'utilisateur à cette dernière***.
- Sinon, un toast signalant l'échec de connexion doit être affiché.

1. Développer le code du bouton ***btnLogin*** pour qu'il puisse faire le travail décrit ci-dessus. **(4)**

```
binding.btnLogin.setOnclickListener {it:View

}
```

2. Dès son apparition sur l'écran, l'activité « **HomeActivity** » doit afficher dans un **TextView** d'id ***txtWelcome*** un message de bienvenue au nom de l'utilisateur envoyé par l'activité d'authentification.
Donner les lignes de code nécessaires permettant de récupérer le nom de l'utilisateur et d'afficher le message de bienvenue dans le TextView. **(2)**

....

....

....

....

....

....

....

....

....

....

....

....

....

3. **HomeActivity** contient un menu d'options contenant 3 items : « Chercher », « A propos » et « Quitter » d'IDs respectifs ***menu_search, menu_about*** et ***menu_quit***.
Ecrire le code de la méthode ***onOptionsItemSelected()*** ci-dessous afin d'implémenter les fonctionnalités suivantes : **(3)**
   - L'option « *Search* » permet de passer à une activité nommée « *SearchActivity* ».
   - L'option « *About* » lance un toast affichant la version de l'application (par exemple, v1.0).
   - L'option « *Quit* » permet de quitter l'application.

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {

    return when(item.itemId) {
```

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

```
        }

}
```

4. « ***SearchActivity*** » contient une zone de texte d'id « ***editSearch*** » et un bouton d'id « ***btnSearch*** ».
À la suite du clic sur le bouton, une recherche de la chaîne saisie dans la zone de texte sera lancée sur Google.
Ecrire le code du bouton « ***btnSearch*** » permettant de lancer cette recherche Google sachant que l'URL correspondante a la forme suivante : **(2)**
« **https://www.google.com/search?q=[*CHAINE_A_CHERCHER*]** »

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

....

BON TRAVAIL